

İSTANBUL BİLGİ UNIVERSITY  
FACULTY of  
SCIENCE AND LETTERS

*Quantum Computation:  
Implementation of Quantum Gates on Linux Clusters*

SUBMITTED By  
Arda Karaduman

In Partial Fulfillment of the Requirements for the Degree of  
Bachelor of Science in Computer Science

Originally submitted: July 25, 2004

Revised: May 21, 2026

Originally Approved By

Chris Stephenson  
Head of Department

Bülent Özel  
Dissertation Supervisor

## Foreword

My original dissertation thesis in 2004 was incomplete. This has been bugging me for a while, so I decided to revamp it using AI.

I stayed true to the state of facts as of 2004 in the historical and introductory chapters, filled in the missing algorithms, sketched the implementation strategy that the parallel simulator should have used, and made the build itself easier to execute in 2026. The exceptions are Sections 8 through 11: the implementation strategy in Section 8 is informed by what the simulator community has converged on since 2004, and the algorithm chapters draw on later expository work (Kitaev’s phase estimation, Beauregard’s  $2n + 3$ -qubit Shor circuit, qHiPSTER’s distributed state-vector design, the Harrow–Hassidim–Lloyd linear-systems algorithm) that did not exist when the original thesis was submitted.

— Arda Karaduman, May 21, 2026

## Abstract

This thesis gives an introduction to **Quantum Computation** and discusses implementation strategies for a **Quantum Computer** simulator on a distributed system. Research into **Quantum Computers** has expanded rapidly since Peter Shor showed that integer factorisation — believed to be classically intractable — admits a polynomial-time quantum algorithm; further quantum algorithms have since been discovered that outperform their classical counterparts on a range of problems. Physical realisation of **Quantum Computers** remains difficult, and machines built to date are limited in size and noisy in operation; as a result, much of the research into **Quantum Computation** algorithms makes use of simulators running on classical hardware. The fundamental difference between classical and quantum computation makes such simulation inherently expensive: the state vector grows exponentially in the number of qubits. This thesis examines the strategies available for simulating a **Quantum Computer** on a parallel **Linux** cluster, identifies the central scaling pitfall of dense-matrix approaches, and presents the in-place sparse-gate strategy that scalable simulators must adopt instead. It then applies that strategy to the three canonical quantum algorithms: the **Quantum Fourier Transform**, Grover’s unstructured search, and Shor’s integer factorisation.

# Contents

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>Foreword</b>                                                       | <b>i</b>   |
| <b>Abstract</b>                                                       | <b>ii</b>  |
| <b>Table of Contents</b>                                              | <b>iii</b> |
| <b>List of Figures</b>                                                | <b>vi</b>  |
| <b>1 Introduction</b>                                                 | <b>1</b>   |
| <b>2 History of Quantum Computation</b>                               | <b>2</b>   |
| <b>3 Models of Computation</b>                                        | <b>4</b>   |
| <b>4 Physical Background</b>                                          | <b>6</b>   |
| 4.1 The four postulates . . . . .                                     | 6          |
| 4.2 Superposition is linearity . . . . .                              | 7          |
| 4.3 Entanglement and coherence are distinct . . . . .                 | 7          |
| 4.4 Why computation must be reversible . . . . .                      | 8          |
| 4.5 The no-cloning theorem . . . . .                                  | 9          |
| <b>5 Mathematical Basics</b>                                          | <b>10</b>  |
| 5.1 Dirac notation and the single-qubit state space . . . . .         | 10         |
| 5.2 The tensor product and the multi-qubit state space . . . . .      | 10         |
| 5.3 Single-qubit gates as operators on a register . . . . .           | 11         |
| 5.4 Unitary representation of arbitrary classical functions . . . . . | 12         |
| <b>6 Quantum Computation</b>                                          | <b>14</b>  |
| 6.1 Unitary evolution as computation . . . . .                        | 14         |
| 6.2 From Hamiltonians to gates . . . . .                              | 15         |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| 6.3       | Measurement and the central challenge . . . . .                 | 15        |
| 6.4       | Practical limitations . . . . .                                 | 16        |
| <b>7</b>  | <b>Simulation</b>                                               | <b>18</b> |
| <b>8</b>  | <b>Parallel Simulation</b>                                      | <b>19</b> |
| 8.1       | Why the dense-matrix approach fails . . . . .                   | 19        |
| 8.2       | The right primitive: in-place sparse gate application . . . . . | 20        |
| 8.3       | Asymptotic comparison . . . . .                                 | 21        |
| 8.4       | Distributed state vector across MPI nodes . . . . .             | 22        |
| 8.5       | Scaling estimate . . . . .                                      | 23        |
| 8.6       | Relation to the original library . . . . .                      | 24        |
| <b>9</b>  | <b>Quantum Fourier Transform</b>                                | <b>25</b> |
| 9.1       | Definition . . . . .                                            | 25        |
| 9.2       | Circuit decomposition . . . . .                                 | 25        |
| 9.3       | Implementation in the simulator . . . . .                       | 26        |
| 9.4       | Inverse QFT . . . . .                                           | 27        |
| 9.5       | Why it matters . . . . .                                        | 27        |
| <b>10</b> | <b>Grover's Algorithm</b>                                       | <b>28</b> |
| 10.1      | The problem . . . . .                                           | 28        |
| 10.2      | The construction . . . . .                                      | 28        |
| 10.3      | Geometric picture . . . . .                                     | 29        |
| 10.4      | Implementation in the simulator . . . . .                       | 29        |
| 10.5      | Worked example . . . . .                                        | 31        |
| <b>11</b> | <b>Shor's Factoring Algorithm</b>                               | <b>32</b> |
| 11.1      | Reduction to order finding . . . . .                            | 32        |
| 11.2      | Order finding via quantum phase estimation . . . . .            | 32        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 11.3 The expensive subroutine: modular exponentiation . . . . . | 33        |
| 11.4 Classical post-processing . . . . .                        | 35        |
| 11.5 Resource requirements . . . . .                            | 36        |
| <b>12 Library Documentation</b>                                 | <b>38</b> |
| 12.1 The quantum register . . . . .                             | 38        |
| 12.2 Gate primitives . . . . .                                  | 39        |
| 12.3 Measurement and inspection . . . . .                       | 40        |
| 12.4 Higher-level routines . . . . .                            | 41        |
| 12.5 Note on the bundled 2004 implementation . . . . .          | 43        |
| <b>13 Conclusion</b>                                            | <b>44</b> |
| <b>References</b>                                               | <b>46</b> |

# List of Figures

|   |                                                                  |    |
|---|------------------------------------------------------------------|----|
| 1 | Dr. Isaac Chuang holding a 7-qubit NMR quantum computer. . . . . | 17 |
|---|------------------------------------------------------------------|----|

# 1 Introduction

Since the first computers, the trend has been to miniaturise these machines in order to increase their efficiency and convenience. But there are clear physical limits to miniaturisation. Benioff [2] proposed that the ultimate limit of miniaturisation would be a quantum mechanical system. Soon afterwards, Deutsch [5] showed that the benefits of a quantum computer were not merely its size and energy savings: an entirely new mode of computation became possible, exploiting entangled quantum particles and quantum parallelism. Following this, Shor [15] discovered a polynomial-time prime factorisation algorithm for a quantum computer, generating great interest in the field. Together with Grover’s algorithm for fast searching of an unsorted database [8], Shor’s algorithm provided the motivation for scientists to investigate the realisation of physical quantum computing devices.

Despite the rapid progress in the theory of quantum computation, experimental implementation of a quantum computer faces major challenges. Although very basic quantum computing devices have been demonstrated using a variety of techniques, the realisation of a useful, large-scale quantum computer in the near future remains uncertain.

For this reason, quantum-computer simulation on classical computers has become a popular topic. Several libraries<sup>\*</sup> and standalone applications have been developed for this purpose, some free,<sup>†‡</sup> some proprietary,<sup>§</sup> and several dedicated languages have appeared.<sup>¶</sup> Because the computational cost of state-vector simulation grows exponentially with the number of qubits, all of these tools eventually hit the same scaling wall. To push this wall back further, this thesis discusses a method for quantum-computer simulation on a distributed cluster. A similar approach has been pursued by Glendinning and Ömer [7].

---

<sup>\*</sup>libquantum (<http://www.enyo.de/libquantum/>)

<sup>†</sup>jaQuzzi (<http://www.eng.buffalo.edu/~phygons/jaQuzzi/jaQuzzi.html>)

<sup>‡</sup>Qudit (<http://www.compsoc.nuigalway.ie/~damo642/QuantumSimulator/>)

<sup>§</sup>Senko’s simulator (<http://www.senko-corp.co.jp/qcs/qcp/qcpe.html>)

<sup>¶</sup>QCL (<http://tph.tuwien.ac.at/~oemer/qcl.html>)

<sup>||</sup>Q language (<http://sra.itc.it/people/serafini/qlang/>)

## 2 History of Quantum Computation

At the turn of the 1980s, working independently, Paul Benioff and Richard Feynman each claimed that quantum computing could be possible. Simulating quantum systems on classical computers requires an exponentially large amount of time, and Feynman's (1982) starting point was exactly this difficulty: he proposed that a quantum system could simulate other quantum systems more efficiently [6]. Benioff (1980) was motivated rather by continuing the trend of miniaturisation in the computer industry [2]: as circuits continue to shrink they will eventually reach the atomic level, where quantum mechanical effects become a major limiting factor, so the only feasible long-term solution was to construct a computer that would make use of those effects rather than fight them.

In 1985 David Deutsch published a paper [5] which proposed a quantum Turing machine, which made use of a before unnoticed property of quantum computers. Though severely underrated at its time, Deutsch's paper would later be considered a landmark in quantum computation theory. In his paper, Deutsch showed that it is possible to perform some tasks on quantum computers much more efficiently than classical computers. Deutsch's idea was to make use of a phenomenon which is known as quantum superposition. An isolated electron, if placed in a superposition, can travel along many different paths through a circuit. If the memory could be placed in a quantum superposition before applying the algorithm, it could be in multiple states simultaneously. So just one run-through the algorithm would calculate all computational paths simultaneously. This idea (Which was to be called *quantum parallelism*) would be the foundation of all quantum algorithms.

The examples in Deutsch's paper were of limited use and due to their probabilistic nature, should be run several times to ensure the correct outcome. But these early hints in the field of quantum computation theory stirred the imaginations of many scientists, and about 10 years later Peter Shor of AT&T labs came up with a factorization algorithm for quantum computers.

In 1994 Peter Shor of AT&T developed a quantum algorithm for integer factorisation

[15] that runs in time polynomial in the number of input bits — super-polynomially faster than any known classical algorithm. This discovery triggered enormous interest in quantum computation, because factorisation is widely believed to be classically hard: the best known classical algorithms (the general number field sieve and its variants) run in sub-exponential but super-polynomial time. The decision version of factoring lies in  $\text{NP} \cap \text{co-NP}$ , which is one reason most complexity theorists do *not* expect it to be NP-complete — such a result would collapse  $\text{NP} = \text{co-NP}$ . Whether factoring is in P classically remains an open problem. These difficulties underpin many modern cryptosystems, most prominently RSA (the Rivest–Shamir–Adleman protocol), which rests on the difficulty of factoring a very large number that is the product of two prime secret keys. Shor’s algorithm shows that quantum computers can factor such numbers in polynomial time, which would compromise the security of RSA and other widely deployed key-exchange protocols once large enough quantum hardware exists.

Only two years later Lov Grover of Bell Labs invented a quantum algorithm for fast searching of an unsorted database [8]. Although Grover’s algorithm gives only a quadratic speed-up over the best classical algorithm — in contrast to Shor’s super-polynomial separation — searching is so foundational a primitive, and Grover’s algorithm so readily plugs in to amplify the efficiency of many existing algorithms, that its discovery carries comparable importance.

These algorithms show the capabilities of a quantum computer, and the main motivation behind the research into quantum computation. It is logical to expect even more powerful algorithms to be discovered in the near future. However, an implementation of a quantum computer which would be capable of convenient applications seems far away. At the time of the original 2004 submission, quantum computers could operate only on a few qubits, and under extreme environmental conditions; the present state of the art is summarised in the next chapter.

### 3 Models of Computation

Classical computers can be modeled by a Turing machine. A *Turing Machine* is an abstract model of computation comprised of an infinitely long sequential tape, a read/write head, a set of symbols, and a set of rules for reading / writing and moving the head based on the symbol under it. These rules are called *functions*, and the symbol under the head is called the *state*. The machine moves from one state to another by executing a matching transition function. If no transition function matches, the machine halts.

The *Church–Turing Thesis* [4, 17] says the following: a Turing machine can compute any function computable by a reasonable physical device. Programming languages can be translated into Turing machines and vice versa. Thus, the computation of any reasonable function can be expressed by a programming language.

The *extended* (or *strong*) Church–Turing thesis goes further than this and asserts that a probabilistic Turing machine can simulate any physical device with at most polynomial overhead. This is not a statement made by Church or Turing in their 1936 papers; it is a later thesis about the resources required for simulation. A *probabilistic Turing Machine* can select among a set of transition functions with varying probability, instead of just one. In the following example, real numbers preceding a state indicate the probability of transitioning to that state.

$$\begin{aligned}
 |0\rangle &\rightarrow 0.5 |1\rangle + 0.5 |2\rangle \\
 |1\rangle &\rightarrow 0.2 |0\rangle + 0.8 |1\rangle \\
 |2\rangle &\rightarrow 0.4 |0\rangle + 0.3 |2\rangle + 0.3 |3\rangle \\
 |3\rangle &\rightarrow 0.3 |0\rangle + 0.7 |2\rangle
 \end{aligned}$$

Such machines can solve some problems efficiently with bounded or negligible error, even when the best known deterministic exact algorithms are much more expensive. Bounded error does not enlarge the class of computable functions — anything a probabilistic Turing machine can decide with bounded error is already computable by a deterministic machine

— but it can dramatically change the *resources* required. Probabilistic computation with negligible error is the right reference class for quantum computation, whose output is also sampled rather than read out exactly.

## 4 Physical Background

Quantum mechanics is a branch of physics which models the behavior of energy and matter at atomic and subatomic scales. Its name comes from *quanta*<sup>\*\*</sup>.

Rather than enumerate informal “theories” of quantum mechanics, the cleanest way to present the subject is through its four postulates. Every result we use in this thesis — superposition, entanglement, the reversibility of computation, measurement statistics — is a consequence of these four statements and not an additional independent principle.

### 4.1 The four postulates

**Postulate 1 (State space).** The state of an isolated quantum system is a unit vector in a complex Hilbert space  $\mathcal{H}$ . For a single qubit,  $\mathcal{H} = \mathbb{C}^2$ , and the state is

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (4.1)$$

Two states that differ only by a global phase  $e^{i\theta}$  are physically indistinguishable.

**Postulate 2 (Evolution).** The evolution of a closed quantum system is unitary. A discrete time step is described by a unitary operator  $U$  acting on the state:

$$|\psi'\rangle = U |\psi\rangle. \quad (4.2)$$

Continuous evolution is governed by the Schrödinger equation  $i\hbar \partial_t |\psi\rangle = H |\psi\rangle$ , whose finite-time solution is  $U = e^{-iHt/\hbar}$ . For computation we usually work with  $U$  directly and treat the underlying Hamiltonian  $H$  as an implementation detail of the physical hardware.

**Postulate 3 (Measurement).** A measurement in the computational basis returns outcome  $x$  with probability  $|\langle x|\psi\rangle|^2$  and projects the state onto  $|x\rangle$ . More generally, a mea-

---

<sup>\*\*</sup>the discrete packets of energy proposed by Max Planck in 1900

surement is described by a collection of operators  $\{M_m\}$  satisfying  $\sum_m M_m^\dagger M_m = I$ ; outcome  $m$  occurs with probability  $\langle \psi | M_m^\dagger M_m | \psi \rangle$ , and after observing  $m$  the state collapses to  $M_m | \psi \rangle / \sqrt{\langle \psi | M_m^\dagger M_m | \psi \rangle}$ .

**Postulate 4 (Composite systems).** The state space of a composite system is the tensor product of the component state spaces. For two qubits,

$$\mathcal{H} = \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4. \quad (4.3)$$

## 4.2 Superposition is linearity

Superposition is the statement that if  $|\psi_1\rangle$  and  $|\psi_2\rangle$  are valid states, then so is any normalised linear combination  $\alpha |\psi_1\rangle + \beta |\psi_2\rangle$ . It is a direct consequence of Postulate 1: state space is a vector space.

This has nothing to do with the Heisenberg uncertainty principle. The uncertainty principle is a statement about *observables*: for any two observables  $A$  and  $B$ ,

$$\Delta A \cdot \Delta B \geq \frac{1}{2} \left| \langle \psi | [A, B] | \psi \rangle \right|. \quad (4.4)$$

When  $A$  and  $B$  do not commute, no quantum state can simultaneously have sharp values for both. Position and momentum are the canonical example. Superposition, in contrast, exists in finite-dimensional Hilbert spaces with no position observable at all (a single qubit being the simplest example). The two ideas live on different shelves and should not be conflated.

## 4.3 Entanglement and coherence are distinct

A two-qubit state is *entangled* if and only if it cannot be written as a tensor product  $|\phi\rangle \otimes |\chi\rangle$  of single-qubit states. The four canonical Bell states are all maximally entangled:

$$|\Phi^\pm\rangle = \frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle), \quad |\Psi^\pm\rangle = \frac{1}{\sqrt{2}}(|01\rangle \pm |10\rangle). \quad (4.5)$$

Measuring the first qubit of  $|\Phi^+\rangle$  yields outcomes that *match* the second qubit's; measuring the first qubit of  $|\Psi^-\rangle$  yields outcomes that *anti-match*. Entanglement is non-classical correlation in general — not specifically anti-correlation, and not specifically the spin-singlet case.

*Coherence* is a different concept entirely. A state is coherent (with respect to a chosen basis) when its density matrix has non-zero off-diagonal elements in that basis. For a single qubit in the state  $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ , the density matrix

$$\rho_+ = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \quad (4.6)$$

is coherent in the  $\{|0\rangle, |1\rangle\}$  basis — the off-diagonal elements record the relative phase between  $|0\rangle$  and  $|1\rangle$ . After decoherence in that basis, the diagonal survives but the off-diagonals vanish:

$$\rho_{\text{dec}} = \frac{1}{2} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \quad (4.7)$$

A single-qubit superposition is coherent but trivially unentangled; one can also have entanglement on noisy mixed states with reduced coherence. They are separate resources. Decoherence — caused by uncontrolled interaction with the environment, sometimes by even a single stray photon — is the process that destroys both.

## 4.4 Why computation must be reversible

Landauer (1961) [12] showed that erasing one bit of information in a system at temperature  $T$  dissipates at least  $k_B T \ln 2$  joules of heat. Bennett (1973) [3] showed that any classical computation can be made logically reversible by retaining intermediate results, so heat dissipation is not fundamentally required for computation.

Quantum computation is reversible by construction: Postulate 2 says that closed-system evolution is unitary, and unitary maps are invertible ( $U^{-1} = U^\dagger$ ). The deeper reason this

matters, beyond thermodynamic prudence, is that non-unitary evolution is what decoherence *looks like* from the outside. A computation that “leaks” information into the environment is, by definition, a computation entangled with environmental degrees of freedom that we cannot track, and tracing them out collapses our superposition. Reversibility is therefore the property a quantum computer must preserve to keep its computational state coherent.

In practice, achieving reversibility requires extensive isolation of the qubit register, and active mitigation through quantum error-correcting codes, which encode each logical qubit redundantly across many physical qubits.

## 4.5 The no-cloning theorem

One result deserves explicit mention because it shapes both quantum algorithms and quantum error correction: there is no unitary  $U$  such that

$$U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle \tag{4.8}$$

for arbitrary  $|\psi\rangle$ . The proof is one line: assume such a  $U$  exists, apply it to two non-orthogonal states  $|\psi\rangle$  and  $|\phi\rangle$ , take inner products on both sides of the resulting identity, and conclude  $\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2$ , which forces  $\langle\psi|\phi\rangle \in \{0, 1\}$  — a contradiction [19]. This is why quantum error correction cannot simply “copy and check” to recover from errors; it must protect the state without ever reading it directly.

In order to explain quantum algorithms and the efficiency gains they achieve, we now need to understand how the abstract qubit register works internally, and how it is described mathematically.

## 5 Mathematical Basics

### 5.1 Dirac notation and the single-qubit state space

Qubits are conventionally represented as *ket* vectors in the Dirac (bra–ket) notation, which is standard in quantum physics. A ket  $|\psi\rangle$  denotes a column vector and its conjugate transpose  $\langle\psi|$  is the bra (a row vector). In quantum computation, the expression  $|0\rangle$  represents the quantum analogue of the classical bit value 0, and  $|1\rangle$  the analogue of 1.

The state of a qubit can then be written as

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad (5.1)$$

where  $\alpha, \beta \in \mathbb{C}$  are the *probability amplitudes* of  $|0\rangle$  and  $|1\rangle$  respectively, and (by Postulate 1)  $|\alpha|^2 + |\beta|^2 = 1$ . The probability of obtaining outcome 0 on a computational-basis measurement is  $|\alpha|^2$ , and of 1 is  $|\beta|^2$ . More generally, an  $n$ -qubit register has  $2^n$  amplitudes whose squared magnitudes sum to 1.

In matrix form the computational-basis states are

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad (5.2)$$

and an arbitrary single-qubit state is the unit vector  $(\alpha, \beta)^T$ .

### 5.2 The tensor product and the multi-qubit state space

The state space of a composite system is the tensor product of the component state spaces (Postulate 4). For matrices  $A$  ( $n \times m$ ) and  $B$  ( $p \times q$ ), the Kronecker (tensor) product is the

$np \times mq$  matrix

$$A \otimes B = \begin{pmatrix} a_{11}B & \cdots & a_{1m}B \\ \vdots & \ddots & \vdots \\ a_{n1}B & \cdots & a_{nm}B \end{pmatrix}. \quad (5.3)$$

Specialised to two column vectors  $\vec{a} \in \mathbb{C}^n$  and  $\vec{b} \in \mathbb{C}^m$ ,

$$\vec{a} \otimes \vec{b} = (a_1b_1, a_1b_2, \dots, a_1b_m, a_2b_1, \dots, a_2b_m, \dots, a_nb_1, \dots, a_nb_m)^T. \quad (5.4)$$

**Convention.** Throughout this thesis we number the qubits of an  $n$ -qubit register  $0, 1, \dots, n-1$  from the least significant bit upward — the natural indexing for code — and write tensor products in *big-endian* order, with the more significant qubit on the left. So for two qubits indexed 1 (more significant) and 0 (less significant), with single-qubit kets  $|a\rangle = (a_0, a_1)^T$  on qubit 1 and  $|b\rangle = (b_0, b_1)^T$  on qubit 0,

$$|a\rangle_1 \otimes |b\rangle_0 = \begin{pmatrix} a_0b_0 \\ a_0b_1 \\ a_1b_0 \\ a_1b_1 \end{pmatrix}. \quad (5.5)$$

Equivalently, the index of basis state  $|x_{n-1}x_{n-2}\dots x_0\rangle$  in the amplitude vector is the integer  $\sum_{k=0}^{n-1} x_k \cdot 2^k$ . Fixing this convention once and using it consistently is essential: changing endianness mid-derivation swaps amplitude indices and silently breaks every matrix–vector multiplication that follows.

### 5.3 Single-qubit gates as operators on a register

If  $U$  is a  $2 \times 2$  unitary and we want to apply it to qubit  $k$  of an  $n$ -qubit register (with qubits indexed  $0, \dots, n-1$  from the least significant bit, which is the convenient indexing for code),

the operator on the full  $2^n$ -dimensional space is

$$U_k = I^{\otimes(n-1-k)} \otimes U \otimes I^{\otimes k}. \quad (5.6)$$

This is a  $2^n \times 2^n$  matrix, but it is extremely sparse: each row has exactly two non-zero entries, in two specific columns. We will see in Section 8 that materialising  $U_k$  as a dense matrix is the single mistake one most wants to avoid in a state-vector simulator — the matrix never needs to be built explicitly.

## 5.4 Unitary representation of arbitrary classical functions

A quantum program transforming an  $n$ -qubit register from input to output state can in principle be written as a  $2^n \times 2^n$  unitary matrix which, when multiplied by the input-state vector, yields the output-state vector. Quantum computation must be information-lossless (and therefore reversible at each gate); the operator must be unitary.

What about classical functions that are not bijections? Any classical function  $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$  can be made reversible — and hence unitary — using the Bennett construction. Append an  $m$ -qubit ancilla register and define  $U_f$  on the entire joint Hilbert space by its action on computational basis states,

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle, \quad (5.7)$$

where  $\oplus$  denotes bitwise XOR. The first register is preserved and the second is shifted by  $f(x)$ , so  $U_f$  is a permutation of computational basis states and therefore a unitary matrix on the joint  $2^{n+m}$ -dimensional space [3]. The familiar special case

$$U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$$

recovers the value of  $f(x)$  in the ancilla register, but the full XOR form is needed to make

$U_f$  unitary on *all* of  $\mathbb{C}^{2^{n+m}}$ , not just on the subspace where the ancilla is  $|0\rangle$ . This is the standard “bit oracle” pattern used by Deutsch–Jozsa, Grover, and Shor (and reappears in Sections 10 and 11). The earlier claim — common in informal expositions — that a unitary exists “whenever  $f$  is one-to-one” is technically insufficient: it is the Bennett ancilla trick that promotes *any* classical function to a unitary, regardless of whether the original  $f$  was injective.

The combined effect of single-qubit gates applied in parallel to disjoint qubits in a register is the tensor product of the individual gate matrices. Combined with the in-place sparse-application algorithm of Section 8, this is all the machinery one needs to simulate any quantum circuit.

## 6 Quantum Computation

We are concerned with using quantum computation to produce solutions to problems considered intractable for classical computers.<sup>††</sup> To see how quantum computers can solve classically intractable problems, we need to examine how they perform computation.

The building block of the quantum computer is the *qubit*, or quantum bit. A qubit is an abstract value, realised by nano-scale properties of physical matter. Qubits have been implemented using ion charge, photon polarisation, nuclear spins of atoms, and nuclear spins of impurities in silicon chips.

Classical bits abstractly represent values of 0 or 1 by the absence or presence of voltage. A qubit, by superposition, represents a linear combination of 0 and 1 simultaneously.

An  $n$ -qubit register holds a superposition over  $2^n$  basis states. Two qubits, for example, hold a superposition over the four states  $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ , each weighted by a complex *probability amplitude*. Probability amplitudes are mathematically analogous to a wave: they can be positive or negative (or in general complex), and they interfere. The classical probability of an outcome is the squared magnitude of its amplitude. Quantum computation manipulates probability amplitudes through unitary gates so that, on measurement, the amplitudes of desired outcomes interfere constructively while those of undesired outcomes interfere destructively.

### 6.1 Unitary evolution as computation

A single qubit lives in the Hilbert space  $\mathbb{C}^2$ ; an  $n$ -qubit register lives in the tensor product

$$\mathcal{H} = \underbrace{\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \dots \otimes \mathbb{C}^2}_{n \text{ factors}} = \mathbb{C}^{2^n}. \quad (6.1)$$

---

<sup>††</sup>An *intractable* problem is one for which execution time grows exponentially as input size increases linearly.

We compute a function  $f$  on these qubits by applying a unitary operator  $U$  to the state vector:

$$|\psi'\rangle = U |\psi\rangle. \quad (6.2)$$

Any classical function can be expressed in this form using the Bennett ancilla construction from the previous section. We can therefore think of  $U$  as a basic quantum gate, performing the role of a classical truth table, and represent it as a unitary matrix — which by definition is reversible, since  $U^{-1} = U^\dagger$ .

## 6.2 From Hamiltonians to gates

To apply a target unitary  $U$  on physical hardware, we find a Hamiltonian  $H$  which solves

$$U = e^{-iHt/\hbar}. \quad (6.3)$$

A solution exists as long as  $U$  is unitary: by the spectral theorem, every unitary matrix can be written as  $e^{iA}$  for some Hermitian  $A$ , so taking  $H = -\hbar A/t$  gives the desired Hamiltonian. From the standpoint of algorithms we can usually treat  $H$  as an implementation detail and reason directly with  $U$ .

By applying  $U$  to a superposition over all  $2^n$  inputs, we effectively apply  $f$  to all of them in parallel — the so-called *quantum parallelism*. This parallelism is the basic resource that quantum algorithms exploit, though as discussed below, exponential parallelism does not directly translate into exponential computational power; algorithms must arrange interference so that useful information survives measurement.

## 6.3 Measurement and the central challenge

Obtaining the computed answer presents a new challenge. The superposition of  $n$  qubits ranges over  $2^n$  possible computational-basis states; we sometimes refer to it loosely as a “wave function”. Measuring this state (Postulate 3) collapses the superposition, selecting one basis

state from the  $2^n$  with probability equal to the squared magnitude of its amplitude. This selection effectively destroys the rest of the computed information. Quantum algorithms must therefore use the interference property of amplitudes to concentrate probability mass onto basis states encoding the correct answer before measurement is performed.

## 6.4 Practical limitations

Quantum computing is not without drawbacks. The exponential parallelism inherent in quantum computation does not, by itself, imply exponential computational power. Practical implementation is hindered by decoherence, caused both by interaction with the environment and by intrinsic decay of quantum states. Quantum error-correcting codes are necessary to mitigate decoherence and error propagation in the real world; these codes require many physical qubits to encode a single logical qubit. Some quantum architectures have impractical environmental requirements (e.g., operating at temperatures near absolute zero), with classical control hardware that does not function well in the same environment.

Because of these difficulties, simulation on classical computers remains essential to algorithm development and analysis. Quantum simulators allow inspection of the full state, accumulation of errors under noise models, and feasibility studies that would otherwise require physical hardware. However, simulating an  $n$ -qubit register on a classical machine requires storing  $2^n$  complex amplitudes, which restricts simulators to relatively small register sizes — the central scaling problem that motivates the parallel implementation discussed later in this thesis.

When this thesis was first submitted in 2004, real quantum computers operated on roughly 5–7 qubits; Shor’s algorithm had been demonstrated on a 7-qubit nuclear-magnetic-resonance machine in 2001 [18]. Two decades on, the picture is very different: superconducting processors with hundreds of physical qubits are now in service.<sup>‡‡</sup> Even so, error correction is still a heavy multiplier on physical-to-logical qubit counts, and classical simulation remains

---

<sup>‡‡</sup>For up-to-date device counts see IBM’s processor catalogue (<https://quantum.cloud.ibm.com/docs/en/guides/processor-types>); as of this revision the Heron r2 carries 156 qubits and the Nighthawk r1 carries 120.



Figure 1: Dr. Isaac Chuang of IBM's Almaden Research Center holding a 7-qubit nuclear-magnetic-resonance quantum computer — a glass tube of specially designed molecules whose nuclear spins act as qubits. Image reproduced at native resolution from the 2004 thesis source files.

essential for algorithm design and verification — which is the practical motivation for the work that follows.

## 7 Simulation

This project aims to produce a quantum-computer simulator that runs on classical computers. Unfortunately, any classical simulator is fundamentally inefficient compared to the quantum hardware it models. Although a quantum computer operates on a different physical paradigm than a classical computer, quantum computers cannot compute anything that classical computers cannot — the gain is in efficiency, not in the set of computable functions. Classical machines can simulate quantum systems to arbitrary precision by storing the state-vector amplitudes and applying the unitary transformations the algorithm prescribes.

Using a classical system to simulate a quantum computer nullifies any speed advantage of the quantum algorithm: most of the structure that quantum computers exploit implicitly must be programmed and manipulated explicitly. Simulating even a moderate quantum algorithm of, say, 100 qubits would require storing  $2^{100}$  (roughly  $10^{30}$ ) complex amplitudes — more than any conceivable classical machine can hold, and even more far beyond what one could rotate by an arbitrary unitary in any reasonable time.

So although a quantum computer can in principle be simulated by a classical computer, the simulation becomes exponentially expensive as the qubit count grows. The memory required to store the state vector and (depending on the formulation) the operator matrices grows exponentially with the number of qubits, which is the bottleneck the parallelisation strategy of the next section is designed to push back as far as possible.

## 8 Parallel Simulation

This section describes the strategy adopted by the present implementation, and an important correction to the strategy proposed in earlier drafts of this thesis. The earlier strategy — distributing rows of a  $2^n \times 2^n$  *program matrix* across nodes and broadcasting the input state vector — is preserved in the bundled C source as a historical artifact, but it is the wrong primitive for a scalable simulator. We explain why, and then describe what the right primitive looks like.

### 8.1 Why the dense-matrix approach fails

The simulator’s primary task is to apply a sequence of quantum gates to a state vector. A naïve formulation treats each gate as a dense  $2^n \times 2^n$  unitary and multiplies it by the  $2^n$ -dimensional state vector. This is the approach adopted by the original library: a master node broadcasts the input state vector to all worker nodes, the program matrix is partitioned by rows, and each worker multiplies its assigned rows by the broadcast vector.

The defect is not in the parallelisation but in the size of the operator being parallelised. The state vector itself is only  $2^n$  complex numbers, each 16 bytes (one complex-double pair) — for  $n = 25$  this is  $2^{29}$  bytes = 512MB, large but tractable on 2004-era hardware; for  $n = 30$ ,  $2^{34}$  bytes = 16GB, comfortably distributable across a small cluster. A general  $2^n \times 2^n$  unitary, by contrast, has  $2^{2n}$  complex entries, i.e.  $2^{2n+4}$  bytes. For  $n = 25$  that is  $2^{54}$  bytes, roughly 16PB; for  $n = 30$ ,  $2^{64}$  bytes, roughly 16EB. The operator cannot be *stored* anywhere, let alone partitioned across nodes. No amount of cluster bandwidth recovers from running out of memory once  $n$  pushes the operator past the cluster’s aggregate RAM.

The original implementation thus solves the wrong problem. Even with perfect parallel speedup, dense matrix–vector multiplication scales as  $\Theta(2^{2n})$  in both memory and runtime per gate, which is exactly the asymptotic the simulator has to defeat in order to be useful.

## 8.2 The right primitive: in-place sparse gate application

Every quantum circuit decomposes into one- and two-qubit gates. A single-qubit gate  $U$  on qubit  $k$  of an  $n$ -qubit register acts non-trivially on only two amplitudes per basis-state pair: those whose indices differ only in bit  $k$ . Equivalently, recalling Equation 5.6 from the previous chapter,

$$U_k = I^{\otimes(n-1-k)} \otimes U \otimes I^{\otimes k} \quad (8.1)$$

is a sparse  $2^n \times 2^n$  matrix with exactly two non-zero entries per row. Rather than constructing this matrix, we apply  $U$  *in place* on the state-vector array by iterating over the  $2^{n-1}$  amplitude pairs the gate couples and applying a  $2 \times 2$  multiplication to each:

```

1 void apply_single_qubit_gate(complex double *amp, int n,
2                             complex double u[2][2], int k) {
3     size_t stride = 1ULL << k;
4     size_t step   = stride << 1;
5     for (size_t base = 0; base < (1ULL << n); base += step) {
6         for (size_t off = 0; off < stride; off++) {
7             size_t i0 = base + off;          /* bit k = 0 */
8             size_t i1 = i0 + stride;         /* bit k = 1 */
9             complex double a0 = amp[i0];
10            complex double a1 = amp[i1];
11            amp[i0] = u[0][0]*a0 + u[0][1]*a1;
12            amp[i1] = u[1][0]*a0 + u[1][1]*a1;
13        }
14    }
15 }
```

Listing 1: Single-qubit gate applied in place on the state vector.

This is  $\Theta(2^n)$  work, not  $\Theta(2^{2n})$ . No matrix beyond the gate's own  $2 \times 2$  is ever materialised. The outer loop visits each amplitude pair exactly once and is trivially parallelisable.

A controlled- $U$  gate with control qubit  $c$  and target qubit  $t$  has the same shape: iterate over amplitude pairs where the control bit is set, and apply  $U$  to each:

```

1 void apply_controlled_gate(complex double *amp, int n,
2                             complex double u[2][2], int c, int t) {
3     size_t cmask = 1ULL << c;
4     size_t tmask = 1ULL << t;
5     for (size_t i = 0; i < (1ULL << n); i++) {
6         if ((i & cmask) && !(i & tmask)) {
7             size_t j = i | tmask;
8             complex double a0 = amp[i];
9             complex double a1 = amp[j];
10            amp[i] = u[0][0]*a0 + u[0][1]*a1;
11            amp[j] = u[1][0]*a0 + u[1][1]*a1;
12        }
13    }
14 }

```

Listing 2: Controlled single-qubit gate.

These two primitives — single-qubit gate and controlled single-qubit gate — are sufficient to construct every algorithm in this thesis, including the QFT, Grover’s algorithm, and Shor’s algorithm presented in the next three sections. This is also the approach that QC-lib [7], QuEST [10], Intel-QS / qHiPSTER [16], and Qiskit Aer [14] all converge on, for the same reasons given above; for textbook background on the underlying postulates and circuit model, see Nielsen and Chuang [13].

### 8.3 Asymptotic comparison

Table 1 summarises the cost difference. The state vector itself is the same size in both approaches; it is the operator and the per-gate work that change. The sparse-gate approach

pays  $\Theta(2^n)$  per single-qubit or two-qubit gate without ever materialising a  $2^n \times 2^n$  matrix. The dense-matrix approach pays  $\Theta(2^{2n})$  per gate *and* has to store the operator. Only a truly dense, arbitrary  $2^n \times 2^n$  unitary — which the algorithms in this thesis never use — forces  $\Theta(2^{2n})$  on the sparse-gate side.

| Operation                     | Dense-matrix approach            | Sparse-gate approach           |
|-------------------------------|----------------------------------|--------------------------------|
| Store state vector            | $\Theta(2^n)$                    | $\Theta(2^n)$                  |
| Store full operator           | $\Theta(2^{2n})$                 | not materialised               |
| Apply one-qubit gate          | $\Theta(2^{2n})$                 | $\Theta(2^n)$                  |
| Apply two-qubit gate          | $\Theta(2^{2n})$                 | $\Theta(2^n)$                  |
| Apply controlled- $U$ gate    | $\Theta(2^{2n})$                 | $\Theta(2^n)$                  |
| Apply arbitrary dense unitary | $\Theta(2^{2n})$                 | $\Theta(2^{2n})$ (unavoidable) |
| Memory wall on a cluster      | $n \approx 20$ regardless of $P$ | scales with aggregate RAM      |

Table 1: Asymptotic cost per operation for an  $n$ -qubit register under the dense-matrix and sparse-gate strategies. The dense-matrix row counts assume the operator is constructed before being applied; the sparse-gate row counts assume in-place pair updates as described in Section 8.

## 8.4 Distributed state vector across MPI nodes

For  $n$  large enough that the state vector exceeds a single node’s memory, we distribute it across  $P = 2^p$  processes. The process of rank  $r$  holds the amplitudes whose global index has the top  $p$  bits equal to  $r$ ; equivalently, the top  $p$  bits of the global index identify the rank and the bottom  $n - p$  bits give the local offset within that rank.

Gates on the low qubits (indices 0 through  $n - p - 1$ ) act entirely within each process — no communication is required. This is the common case: with  $P = 16$  on a 30-qubit simulation, 26 of the 30 qubits are local to each rank.

Gates on the high qubits (indices  $n - p$  through  $n - 1$ ) require communication. For a single-qubit gate on qubit  $k \geq n - p$ :

1. Each process pairs with the process whose rank differs from its own by exactly bit  $k - (n - p)$ .

2. Each pair exchanges its amplitude array (in chunks if necessary, to bound the transient memory footprint).
3. After the exchange, each process holds the amplitudes it needs to apply the local  $2 \times 2$  multiplication.
4. Optionally, “relabel” the qubits so that the global qubit being operated on becomes local for subsequent gates, allowing several global-qubit gates to be batched into a single permutation-and-exchange step.

The communication pattern is a pairwise exchange of  $\Theta(2^{n-p})$  amplitudes per process per global-qubit gate, with one hop on most cluster topologies (and at most  $\log P$  on a hypercube, which the original library targeted). This is the standard distributed state-vector pattern [16].

In the original library’s design, the existing `broadcast_matrix` primitive would be replaced with a pairwise `MPI_Sendrecv`:

```

1 void exchange_amplitudes(complex double *amp, size_t local_size,
2                           int partner_rank) {
3     complex double *buf = malloc(local_size * sizeof *buf);
4     MPI_Sendrecv(amp, local_size, MPI_C_DOUBLE_COMPLEX, partner_rank, 0,
5                 buf, local_size, MPI_C_DOUBLE_COMPLEX, partner_rank, 0,
6                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
7     /* amp + buf together now contain the qubit pair this gate couples */
8     free(buf);
9 }

```

Listing 3: Pairwise amplitude exchange used for gates on global qubits.

## 8.5 Scaling estimate

For a circuit of  $G$  gates on  $n$  qubits distributed across  $P$  processes, the resource bounds are:

- Local-gate work per process:  $\Theta(G \cdot 2^n / P)$  complex multiplications.
- Communication per global-qubit gate:  $\Theta(2^n / P)$  complex numbers exchanged with one partner process.
- Memory per process:  $16 \cdot 2^n / P$  bytes.

This scales. A modern cluster with 64 nodes at 256 GiB RAM each has  $64 \times 256 \text{ GiB} = 16 \text{ TiB}$  of aggregate memory, which is exactly the size of a 40-qubit complex-double state vector ( $2^{40} \times 16 \text{ bytes}$ ). The cluster therefore *can* hold a 40-qubit register in principle, but the raw fit leaves no headroom for OS overhead, MPI buffers, the transient second slice that pairwise exchange allocates during a global-qubit gate, allocator slack, logging, or any temporary that gather/dump operations introduce. Practical 40-qubit runs require either fewer qubits, more memory per node, single-precision amplitudes (halving the state-vector size at the cost of accumulated rounding), careful chunking, or out-of-core / GPU-aware techniques. The dense-matrix approach, by contrast, hits a memory wall at roughly  $n = 20$  *regardless of cluster size*, because the bottleneck is materialising the operator rather than distributing it.

## 8.6 Relation to the original library

The bundled C source under `implementation/original/` implements the historical dense-matrix strategy: a `matrix` struct of `__complex__ double**`, allocated row-major; `tensor_product` and `dot_product` on dense matrices; and `send_matrix` / `get_matrix` / `broadcast_matrix` for partitioning the program matrix across the LAM/MPI cluster. We have left this source as-is for historical reference; the next chapter documents the API that replaces it in the bundled `implementation/c/` library and its `implementation/go/` and `implementation/python/` siblings.

## 9 Quantum Fourier Transform

The Quantum Fourier Transform (QFT) is the workhorse subroutine of quantum computing. Anywhere a quantum algorithm benefits from period finding, phase estimation, or exploitation of hidden-subgroup structure, a QFT appears. Shor’s factoring algorithm is the most famous example, but the QFT also underlies Kitaev’s phase estimation [11], the Harrow–Hassidim–Lloyd (HHL) linear-systems algorithm [9], and most algebraic quantum algorithms.

### 9.1 Definition

The QFT on  $n$  qubits is the unitary that acts on computational-basis states as

$$\text{QFT } |x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i xy/N} |y\rangle, \quad N = 2^n. \quad (9.1)$$

It is the discrete Fourier transform of the amplitude vector, applied as a unitary in  $O(n^2)$  gates rather than the classical  $O(N \log N)$  arithmetic operations.

### 9.2 Circuit decomposition

Write the integer  $x$  in binary as  $x = x_1x_2 \dots x_n$  (most-significant bit first). The QFT factors into a tensor product:

$$\text{QFT } |x_1x_2 \dots x_n\rangle = \frac{1}{2^{n/2}} \bigotimes_{j=1}^n \left( |0\rangle + e^{2\pi i 0.x_jx_{j+1} \dots x_n} |1\rangle \right), \quad (9.2)$$

where  $0.x_jx_{j+1} \dots x_n$  denotes the binary fraction  $\sum_{\ell=0}^{n-j} x_{j+\ell} \cdot 2^{-\ell-1}$ . Each output qubit ends up in a single-qubit state whose relative phase depends on a suffix of the input bits.

This factorisation yields a circuit constructed from two primitives:

- the Hadamard gate  $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ ;

- controlled phase rotations  $R_k = \text{diag}(1, e^{2\pi i/2^k})$ .

The structure is: for each qubit  $j$  from 1 to  $n$ , apply  $H$  on qubit  $j$ , then for each later qubit  $k > j$  apply a controlled- $R_{k-j+1}$  with control  $k$  targeting  $j$ . Finish with a bit-reversal swap of all qubits, which arises because the factorisation places amplitudes in reverse bit order.

### 9.3 Implementation in the simulator

In the sparse-gate API of Section 8, the QFT is a short double loop:

```

1 void apply_qft(qreg *q, int start, int n) {
2     for (int j = 0; j < n; j++) {
3         int target = start + (n - 1 - j);
4         apply_h(q, target);
5         for (int k = j + 1; k < n; k++) {
6             int control = start + (n - 1 - k);
7             double theta = 2.0 * M_PI / (double)(1ULL << (k - j + 1));
8             apply_controlled_phase(q, control, target, theta);
9         }
10    }
11    /* bit-reversal swaps */
12    for (int i = 0; i < n / 2; i++) {
13        apply_swap(q, start + i, start + n - 1 - i);
14    }
15 }

```

Listing 4: QFT on  $n$  qubits starting at register offset `start`.

The gate count is  $n$  Hadamards plus  $n(n-1)/2$  controlled phases plus  $n/2$  swaps, i.e.,  $\Theta(n^2)$  gates in total. Each gate costs  $\Theta(2^n)$  work in the simulator, giving a total simulator cost of  $\Theta(n^2 \cdot 2^n)$ . The classical fast Fourier transform on the same  $2^n$ -amplitude array would cost  $\Theta(n \cdot 2^n)$ ; the simulated QFT is therefore slower than the classical FFT by a factor of

$n$ . Of course, on physical quantum hardware the QFT acts on all  $2^n$  amplitudes in  $\Theta(n^2)$  gate time, with no enumeration of the amplitude array required, which is the whole point.

## 9.4 Inverse QFT

The inverse QFT, denoted  $\text{QFT}^\dagger$ , is required by phase estimation (and therefore by Shor's algorithm in the next section). It is obtained by running the QFT circuit in reverse with all phase rotation angles negated. In the implementation, this means iterating the outer loop from  $j = n - 1$  down to 0 and applying  $R_k^\dagger$  instead of  $R_k$ .

## 9.5 Why it matters

The QFT is the unitary that turns periodicity into measurable phase. Given a state whose amplitudes are periodic in the basis-state index with period  $r$ , the QFT produces a state whose amplitudes are concentrated near multiples of  $N/r$ . Measuring then yields a good rational approximation to  $s/r$  for some integer  $s$ , from which  $r$  can be recovered by continued fractions. This is the eigenvalue-extracting subroutine that Shor's algorithm uses to factor integers, and that Kitaev's phase estimation uses to extract eigenvalues of arbitrary unitaries.

## 10 Grover's Algorithm

### 10.1 The problem

Given a Boolean function  $f : \{0, 1\}^n \rightarrow \{0, 1\}$  implemented as a quantum oracle, find an  $x$  with  $f(x) = 1$ . Classically, with no structure assumed on  $f$ , this requires  $\Theta(N)$  oracle queries where  $N = 2^n$ . Grover (1996) [8] showed it takes only  $\Theta(\sqrt{N})$  quantum queries — a quadratic speedup. The speedup is provably optimal for unstructured search.

### 10.2 The construction

The oracle is a *phase oracle*: it applies a  $-1$  phase to the marked basis states,

$$O_f |x\rangle = (-1)^{f(x)} |x\rangle. \quad (10.1)$$

The bit-oracle of the previous chapter can be converted into a phase oracle using a single ancilla qubit prepared in the state  $(|0\rangle - |1\rangle)/\sqrt{2}$  via the standard phase-kickback trick.

Initialise the register in the uniform superposition

$$|s\rangle = H^{\otimes n} |0 \dots 0\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle. \quad (10.2)$$

The Grover iteration is the unitary

$$G = (2|s\rangle\langle s| - I) O_f. \quad (10.3)$$

The first factor,  $2|s\rangle\langle s| - I$ , is the *diffusion operator*: it inverts every amplitude about the mean of all amplitudes. It admits the decomposition

$$2|s\rangle\langle s| - I = H^{\otimes n} (2|0 \dots 0\rangle\langle 0 \dots 0| - I) H^{\otimes n}, \quad (10.4)$$

and  $2|0\dots 0\rangle\langle 0\dots 0| - I$  is a phase flip on the all-zeros component, equivalent up to global sign to a multi-controlled- $Z$  with all controls negated.

### 10.3 Geometric picture

Let  $|w\rangle$  be the equal superposition of all marked basis states and  $|w^\perp\rangle$  the equal superposition of all unmarked states. The initial state  $|s\rangle$  lies in the two-dimensional plane spanned by  $|w\rangle$  and  $|w^\perp\rangle$ . The oracle  $O_f$  is a reflection about  $|w^\perp\rangle$ , and the diffusion is a reflection about  $|s\rangle$ . Two reflections compose into a rotation, so each Grover iteration rotates  $|s\rangle$  by angle  $2\theta$  in this plane, where

$$\sin \theta = \sqrt{M/N} \quad (10.5)$$

and  $M$  is the number of marked items. After  $k$  iterations, the probability of measuring a marked state is  $\sin^2((2k+1)\theta)$ , maximised at

$$k_{\text{opt}} = \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rfloor. \quad (10.6)$$

Too few iterations and the amplitude has not rotated all the way into the marked subspace; *too many* and it rotates past it. This is one of the few quantum algorithms with a tight optimal stopping point, and stopping at  $k_{\text{opt}}$  gives a success probability close to 1.

### 10.4 Implementation in the simulator

```

1  typedef void (*oracle_fn)(qreg *q);    /* user-supplied phase oracle */
2
3  void apply_grover(qreg *q, int n, oracle_fn oracle, int iterations) {
4      /* uniform superposition */
5      for (int i = 0; i < n; i++) apply_h(q, i);
6
7      for (int iter = 0; iter < iterations; iter++) {
```

```

8      oracle(q);
9
10     /* diffusion: H, then phase flip on |0..0>, then H */
11     for (int i = 0; i < n; i++) apply_h(q, i);
12     for (int i = 0; i < n; i++) apply_x(q, i);
13     apply_multi_controlled_z(q, n);    /* phase flip on |1..1> */
14     for (int i = 0; i < n; i++) apply_x(q, i);
15     for (int i = 0; i < n; i++) apply_h(q, i);
16 }
17 }

```

Listing 5: Grover’s algorithm: uniform-superposition initialisation followed by  $k$  iterations of oracle + diffusion.

The multi-controlled- $Z$  on  $n$  qubits is, in a state-vector simulator, essentially free — one amplitude needs its sign flipped:

```

1 void apply_multi_controlled_z(qreg *q, int n) {
2     size_t target = (1ULL << n) - 1;    /* |1..1> */
3     if (target_is_local(q, target))
4         q->amp[local_index(q, target)] *= -1.0;
5 }

```

Listing 6: Multi-controlled- $Z$  implementation in the simulator.

On physical hardware this gate is not free: it decomposes into  $\Theta(n)$  Toffoli gates with  $\Theta(n)$  ancillas, which is the actual cost of executing Grover’s algorithm on a real machine. The simulator gets to cheat here precisely because it has direct access to the amplitude array.

## 10.5 Worked example

For  $N = 2^{20}$  and a single marked item, the optimal number of iterations is  $\lfloor \frac{\pi}{4} \sqrt{2^{20}} \rfloor = 804$ . Classical search would expect on the order of  $N/2 \approx 524,000$  queries to find it. The simulator runs all 20 qubits in  $\sim 16$  MB of state-vector memory — well within a single node, no cluster needed. For larger registers, the distributed state-vector strategy of Section 8 extends Grover simulation up to the cluster’s aggregate memory ceiling.

## 11 Shor's Factoring Algorithm

Shor's algorithm [15] is the most famous quantum algorithm: it factors integers in polynomial time, breaking RSA in principle. The full algorithm reduces integer factoring to a quantum subroutine for finding the multiplicative order of a random element, with the order-finding subroutine itself reducing to quantum phase estimation built on the QFT of the previous section.

### 11.1 Reduction to order finding

To factor a composite integer  $N$ , pick a random  $a$  with  $\gcd(a, N) = 1$ . Find the multiplicative order of  $a$  modulo  $N$  — the smallest positive integer  $r$  with  $a^r \equiv 1 \pmod{N}$ . Then:

- if  $r$  is odd, restart with a new  $a$ ;
- if  $a^{r/2} \equiv -1 \pmod{N}$ , restart;
- otherwise,  $\gcd(a^{r/2} \pm 1, N)$  are non-trivial factors of  $N$ .

The probability of obtaining a useful  $r$  is bounded below by a constant: Shor showed it is at least  $1/2$  for  $N$  a product of two distinct odd primes, so a constant number of random choices of  $a$  suffice in expectation. The classical post-processing (gcds and the continued-fraction step below) is trivial. The hard part — the part for which we need a quantum computer — is finding  $r$ .

### 11.2 Order finding via quantum phase estimation

Define the unitary  $U_a$  on  $n = \lceil \log_2 N \rceil$  qubits by

$$U_a |y\rangle = |ay \bmod N\rangle. \quad (11.1)$$

The eigenvalues of  $U_a$  are  $e^{2\pi i s/r}$  for  $s = 0, 1, \dots, r-1$ . Quantum phase estimation extracts a (good rational approximation to a) random one of these eigenvalues, and the continued-fraction expansion recovers the denominator  $r$ .

The circuit uses two registers:

- a *counting* register of  $t = 2n + 1$  qubits, initialised to  $|0\rangle^{\otimes t}$ ;
- a *target* register of  $n$  qubits, initialised to  $|1\rangle$ .

The algorithm proceeds:

1. Apply  $H^{\otimes t}$  to the counting register, producing the uniform superposition.
2. For each counting qubit  $j = 0, 1, \dots, t-1$ , apply a controlled- $U_a^{2^j}$  from counting qubit  $j$  to the target register. This entangles the counting register with  $|a^x \bmod N\rangle$  in the target.
3. Apply the inverse QFT (Section 9) to the counting register.
4. Measure the counting register to obtain an integer  $c \in \{0, 1, \dots, 2^t - 1\}$ .
5. Compute  $r$  from  $c/2^t$  via the continued-fraction expansion.

After step 2 the joint state is

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |a^x \bmod N\rangle. \quad (11.2)$$

The inverse QFT in step 3 concentrates probability mass on values of  $c$  such that  $c/2^t \approx s/r$  for some integer  $s \in \{0, 1, \dots, r-1\}$ , and the continued-fraction algorithm recovers  $r$  from this rational approximation with high probability.

### 11.3 The expensive subroutine: modular exponentiation

Implementing the controlled- $U_a^{2^j}$  gates is the dominant cost of Shor's algorithm. The standard construction builds quantum ripple-carry adders, lifts those to modular addition, lifts

modular addition to modular multiplication, and finally lifts modular multiplication to modular exponentiation. The most efficient known circuits use  $O(n^3)$  gates and dominate the algorithm's total gate count. The Beauregard  $2n+3$ -qubit circuit [1] is the standard textbook construction.

In a state-vector simulator, there are two options.

**Honest implementation.** Build the modular exponentiation circuit from the same quantum adder primitives a physical machine would use. Useful for studying noise, gate counts, and ancilla overhead. The Beauregard circuit and its variants take this route.

**Simulator shortcut.** Since the simulator has direct access to all amplitudes, we can update the target register's basis-state index classically for each non-zero amplitude in the counting register. The simulated quantum state is identical to the one produced by the honest circuit, but the time and ancilla costs are dramatically lower:

```

1  /* Note: this version assumes a single-process simulator: new_global may
2   * fall on a different rank than idx, but local_of() here treats the
3   * whole state vector as local. A distributed version must bucket the
4   * (new_global, amp[idx]) pairs by destination rank, exchange them with
5   * an MPI_Alltoallv, then accumulate into new_amp on each rank.      */
6  /* mod_pow: uint64_t mod_pow(uint64_t base, uint64_t exp, uint64_t mod); */
7  void apply_modular_exp(qreg *q, int counting_start, int t,
8                          int target_start, int n,
9                          uint64_t a, uint64_t N) {
10     complex double *new_amp = calloc(q->local_size, sizeof *new_amp);
11     for (size_t idx = 0; idx < q->local_size; idx++) {
12         if (cabs(q->amp[idx]) < 1e-15) continue;
13         size_t global = global_index(q, idx);
14         uint64_t x      = (global >> counting_start) & ((1ULL << t) - 1);
15         uint64_t y      = (global >> target_start)   & ((1ULL << n) - 1);

```

```

16      /* Pass-through for y >= N keeps the map a permutation over the
17      * full 2^n target register; without it, reserved states y >= N
18      * would all collapse into [0, N), breaking reversibility.    */
19      uint64_t y_new = (y < N) ? (y * mod_pow(a, x, N)) % N : y;
20      size_t new_global = (global
21                          & ~(((1ULL << n) - 1) << target_start))
22                          | ((size_t)y_new << target_start);
23      new_amp[local_of(q, new_global)] += q->amp[idx];
24  }
25  memcpy(q->amp, new_amp, q->local_size * sizeof *new_amp);
26  free(new_amp);
27  }

```

Listing 7: Modular exponentiation, single-process simulator shortcut. Iterates over basis states, classically computes  $a^x \bmod N$ , and rewrites the target register. See note below for the distributed case.

The listing as written assumes the entire state vector is held on a single process; under the distributed layout of Section 8,  $U_a^x$  permutes basis-state indices in a way that can move amplitude across rank boundaries, so a distributed implementation must perform an all-to-all redistribution of  $(\text{new\_global}, q \rightarrow \text{amp}[\text{idx}])$  pairs — typically by bucketing per destination rank and exchanging with `MPI_Alltoallv` — before accumulating into `new_amp`. This shortcut is dishonest as a model of physical execution — it tells us nothing about real gate count or noise — but it produces the correct quantum state for studying the information-theoretic behaviour of the algorithm, and runs in seconds where the honest circuit would take hours.

## 11.4 Classical post-processing

After measurement we have  $c \in \{0, \dots, 2^t - 1\}$ . Compute the continued-fraction expansion of  $c/2^t$  and find the convergent  $p/q$  with  $q < N$  and  $|c/2^t - p/q| < 1/(2 \cdot 2^t)$ . If  $q$  is the

true order of  $a$  modulo  $N$ , we are done; otherwise verify  $a^q \equiv 1 \pmod{N}$ , and if not, restart the quantum portion of the algorithm. The continued-fraction step succeeds with constant probability per quantum run, so a constant expected number of repetitions suffices.

## 11.5 Resource requirements

To factor an  $n$ -bit integer  $N$ , Shor’s algorithm requires approximately  $2n + 1$  counting qubits plus  $n$  target qubits plus  $\Theta(n)$  ancillas for modular arithmetic — roughly  $5n$  qubits in total. For RSA-2048 this is about 10,000 logical qubits; under realistic surface-code overhead the physical qubit count rises to millions. This explains why factoring real-world RSA keys remains out of reach despite the algorithm being more than three decades old.

In simulation, the qubit count is the binding constraint, not the gate count, and the exact count depends on whether one simulates the *honest* modular-arithmetic circuit (counting + target + arithmetic ancillas,  $\approx 5n$  qubits in total) or applies a *shortcut* that replaces the modular-exponentiation subcircuit with an in-place permutation of state-vector amplitudes, giving  $\approx 3n + 1$  qubits (counting register  $t = 2n + 1$  plus target register  $n$ , no arithmetic ancillas required). The bundled `implementation/c/`, `implementation/go/`, and `implementation/python/` libraries all use the shortcut: `apply_modular_exp` computes  $|x\rangle|y\rangle \mapsto |x\rangle|y \cdot a^x \bmod N\rangle$  classically and then writes the result back as a single bulk permutation of the amplitude array. This is faithful to the abstract specification of the gate but is not a quantum circuit; an honest construction would unroll it into  $O(n^3)$  elementary gates with  $\Theta(n)$  ancillas [1].

- Honest-circuit model: factoring  $N = 15$  ( $n = 4$ ) requires roughly  $5n = 20$  qubits (16 MB of state-vector memory — fits on a phone). Factoring  $N = 21$  ( $n = 5$ ) requires  $\sim 25$  qubits (512 MB — fits on a laptop). Factoring a 32-bit number requires  $\sim 160$  qubits, far beyond what state-vector simulation on *any* conceivable classical hardware can reach.

- Shortcut model (as bundled): factoring  $N = 15$  requires  $3n + 1 = 13$  qubits ( $n = 4$  target bits plus  $t = 2n + 1 = 9$  counting bits). Factoring  $N = 21$  requires  $3n + 1 = 16$  qubits; this is the configuration exercised by the `RUN_SHOR_21=1` gated test in all three implementations.

Either way, the simulation budget grows exponentially with  $n$  in memory, which is the asymptotic gap that motivated the entire field — and the reason why simulation is fundamentally a tool for algorithm design and small-scale validation, not a substitute for a physical quantum computer.

## 12 Library Documentation

This section documents the simulator API that realises the sparse-gate strategy of Section 8. The API is implemented in the bundled `implementation/c/` directory (a C11 + OpenMPI 5.x library); a coverage matrix linking each claim in this chapter to the corresponding source file lives at `implementation/c/assessment.md`. Two sibling implementations using the same public surface accompany it: a pure-Go goroutine-parallel port under `implementation/go/` and a PyTorch tensor-ops port under `implementation/python/`. The historical 2004 source, which implements the older dense-matrix approach critiqued in Section 8, is preserved separately under `implementation/original/`.

### 12.1 The quantum register

A quantum register is represented by a public struct holding a local slice of the distributed state vector together with the MPI bookkeeping that identifies which slice this process owns. The fields below are intentionally part of the API surface so that test code, dump utilities, and tight inner loops can read them without going through accessors; the contract for callers is that the lifecycle and state-mutating operations all run through the library functions listed later in this chapter.

```

1  typedef struct {
2      complex double *amp;          /* local slice of the state vector */
3      int    n_qubits;             /* total number of qubits (global) */
4      int    rank, n_procs;        /* MPI rank and process count      */
5      size_t local_size;           /* 2^n_qubits / n_procs amplitudes */
6  } qreg;
```

**Construction.** `qreg_create` allocates a register on the given MPI communicator and zero-initialises the amplitude array:

---

```
1 qreg *q = qreg_create(20, MPI_COMM_WORLD);
```

**Initialisation.** `qreg_init_basis` sets the register to a definite computational basis state  $|k\rangle$ :

```
1 qreg_init_basis(q, 0); /* sets the register to |0...0> */
2 qreg_init_basis(q, 3); /* sets it to |...011>          */
```

**Destruction.** `qreg_destroy` frees all owned memory.

## 12.2 Gate primitives

Single-qubit gates apply a  $2 \times 2$  unitary in place on the local amplitude slice, using the sparse-iteration pattern of Section 8. The library provides direct wrappers for the standard one-qubit gates and a generic primitive for user-supplied  $2 \times 2$  matrices:

```
1 void apply_h      (qreg *q, int target);          /* Hadamard */
2 void apply_x      (qreg *q, int target);          /* Pauli-X   */
3 void apply_y      (qreg *q, int target);          /* Pauli-Y   */
4 void apply_z      (qreg *q, int target);          /* Pauli-Z   */
5 void apply_s      (qreg *q, int target);          /* diag(1,i) */
6 void apply_t      (qreg *q, int target);          /* diag(1,e^{i pi/4}) */
   /*
7 void apply_rx     (qreg *q, int target, double theta); /* exp(-i theta X /
   2) */
8 void apply_ry     (qreg *q, int target, double theta); /* exp(-i theta Y /
   2) */
9 void apply_rz     (qreg *q, int target, double theta); /* exp(-i theta Z /
   2) */
```

```

10 void apply_phase (qreg *q, int target, double theta); /* diag(1,e^{i theta
    }) */
11 void apply_u      (qreg *q, int target, complex double u[2][2]);

```

Two-qubit controlled gates take a control qubit and a target qubit:

```

1 void apply_cnot      (qreg *q, int control, int target);
2 void apply_cz        (qreg *q, int control, int target);
3 void apply_controlled_phase(qreg *q, int control, int target, double theta);
4 void apply_cu        (qreg *q, int control, int target,
5                        complex double u[2][2]);
6 void apply_swap      (qreg *q, int a, int b);

```

For each call, the library inspects whether the target/control qubits are local to this process. If both are local (the common case — see Section 8) the gate is applied with no communication; otherwise the library transparently performs the pairwise amplitude exchange and then the local update.

**Multi-controlled.** Two convenience wrappers cover the common case of a gate conditioned on all qubits in a register or on an arbitrary subset:

```

1 void apply_multi_controlled_z(qreg *q, int n);
2 void apply_multi_controlled_x(qreg *q, const int *controls,
3                                int n_controls, int target);

```

`apply_multi_controlled_z` flips the sign of  $|1\dots 1\rangle$  across all  $n$  qubits and is the standard Grover diffusion ingredient; `apply_multi_controlled_x` applies  $X$  to `target` only when every qubit in `controls` is in state  $|1\rangle$ .

## 12.3 Measurement and inspection

`measure_all` collapses the state and returns the observed basis state on every process:

```
1 size_t outcome = measure_all(q);
```

`prob_of` returns  $|\langle k|\psi\rangle|^2$  for a chosen basis state  $k$  without disturbing the register:

```
1 double p = prob_of(q, 42);
```

`measure_qubit` performs a partial measurement of a single qubit and projects the state accordingly:

```
1 int bit = measure_qubit(q, 3);
```

`sample_distribution` draws `shots` independent samples from  $|\psi|^2$  without collapsing the register, returning the outcomes in `out`; the helpers `qreg_clone`, `qreg_norm`, and `qreg_dump` support testing and diagnostics:

```
1 void sample_distribution(const qreg *q, size_t *out, int shots);
2 qreg *qreg_clone      (const qreg *q);
3 double qreg_norm      (const qreg *q);
4 void qreg_dump         (const qreg *q, FILE *f);
```

## 12.4 Higher-level routines

The chapters on the QFT (Section 9), Grover (Section 10), and Shor (Section 11) build on the primitives above. The QFT is a double loop of Hadamards and controlled phases; Grover is a uniform-superposition initialisation followed by alternating oracle and diffusion calls; Shor combines phase estimation, controlled modular exponentiation, and inverse QFT. Each is exposed as a top-level function in the library so applications need not assemble the circuit by hand. The Grover oracle is supplied as a callback together with an opaque `user` pointer carrying its context, and the Shor period-finder returns the recovered period `r` together with the raw measurement `measured_c` in a small result struct:

```
1 typedef void (*oracle_fn)(qreg *q, void *user);
```

```

2
3 typedef struct {
4     uint64_t r;          /* candidate period from continued      *
5                          * fractions; verify  $a^r \bmod N == 1$       *
6                          * before treating as the true order      */
7     uint64_t measured_c; /* raw counting-register outcome      */
8 } shor_period_result;
9
10 void apply_qft      (qreg *q, int start, int n);
11 void apply_qft_inverse (qreg *q, int start, int n);
12 void apply_grover   (qreg *q, int n_qubits, oracle_fn oracle,
13                     void *user, int iterations);
14 shor_period_result
15     apply_shor_period (qreg *q, int counting_start, int t,
16                     int target_start, int n,
17                     uint64_t a, uint64_t N);

```

A top-level wrapper composes random base selection, period finding, and classical post-processing into a single call that attempts to factor  $N$  up to `max_attempts` times:

```

1 typedef struct {
2     uint64_t p, q; /* non-trivial factors (1, N on failure) */
3     int attempts; /* number of period-finding rounds used */
4 } shor_factor_result;
5
6 shor_factor_result shor_factor(uint64_t N, int max_attempts);

```

## 12.5 Note on the bundled 2004 implementation

The C source under `implementation/original/` (files `matrix.c`, `parallel.c`, `qubit.c`, `standart.c`) implements a different API based on a dense matrix struct of type `__complex__ double**`:

```

1  typedef struct {
2      int row_size;
3      int column_size;
4      __complex__ double ** value;
5  } matrix;

```

with operations `create_matrix`, `tensor_product`, `dot_product`, and the MPI primitives `send_matrix`, `get_matrix`, and `broadcast_matrix`. This code targets the original dense-matrix strategy critiqued in Section 8 and is retained here for historical reference. The API documented above is the design that replaces it; the bundled `implementation/c/` library is a working v1 of that design, with limitations documented in its README and assessment matrix (chief among them: per-MPI-message counts are limited to `INT_MAX`, distributed multi-controlled-X is not yet supported, and Shor coverage is exercised against  $N = 15$  and  $N = 21$ ).

## 13 Conclusion

Until quantum computers with enough qubits and low enough error rates to run useful applications become available, classical simulators will remain the primary venue for the development and analysis of quantum algorithms. The fundamental difficulty in simulating quantum systems on classical hardware — the exponential growth of the state vector with the number of qubits — motivates the use of parallel computation, and the simulation of small-to-medium registers across distributed clusters is a continuing area of active research.

This thesis began by introducing the four postulates of quantum mechanics, then built up the mathematical machinery (Dirac notation, the tensor product, unitary evolution, measurement) needed to describe quantum computation. It surveyed the historical landmarks of the field — Benioff’s quantum-mechanical Hamiltonian model, Feynman’s quantum simulator proposal, Deutsch’s quantum Turing machine, Shor’s factoring algorithm, Grover’s search algorithm — and presented the three canonical algorithms in enough detail to be implementable: the Quantum Fourier Transform, Grover’s unstructured search, and Shor’s order-finding subroutine.

The implementation discussion was the part of the thesis that needed the most revision. The original strategy of distributing rows of a  $2^n \times 2^n$  dense program matrix across MPI nodes was, as Section 8 explains, the wrong primitive: the operator itself exceeds any conceivable memory budget for  $n \gtrsim 20$ , so no parallelisation strategy can rescue it. The right primitive is in-place sparse gate application directly on the state vector, with single-qubit gates costing  $\Theta(2^n)$  work rather than  $\Theta(2^{2n})$ , and with distributed simulation arranged so that gates on low qubits cost no communication while gates on high qubits induce one pairwise exchange. A first implementation of this corrected design is now bundled with the thesis under `implementation/c/`: a C11 + OpenMPI 5.x library that exposes the API documented in Section 12, with QFT, Grover, and Shor primitives on top, tested at NP=1,2,4 in the default loop and at NP up to 8 in the larger gated suite. Two sibling ports (`implementation/go/`, `goroutine-parallel`; `implementation/python/`, PyTorch tensor-ops) cover the same public

surface via different parallelism models. The bundled 2004 C source remains as a historical record of the dense approach under `implementation/original/`.

The current v1 implementations are deliberately small. Several extensions are left as future work: chunked large-count MPI communication so individual messages can exceed `INT_MAX` complex amplitudes; distributed multi-controlled gates (currently local-only on `/c`); noise models and density-matrix evolution; stabilizer-formalism and tensor-network shortcuts for circuits where they apply; GPU offload beyond the PyTorch port; and realistic modular-arithmetic circuit constructions (Beauregard, Draper, etc.) replacing the in-place permutation shortcut used in the bundled Shor implementations. The complete library, with the QFT, Grover, and Shor circuits exposed as primitives, has proven useful in this revision for research into quantum algorithms — and for teaching the subject without first requiring access to physical hardware. The Fourier transform and the algorithms that depend on it, in particular, are no longer optional features but the load-bearing core of the field.

## References

- [1] Stephane Beauregard. Circuit for Shor’s algorithm using  $2n + 3$  qubits. *Quantum Information and Computation*, 3(2):175–185, 2003.
- [2] Paul Benioff. The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics*, 22:563–591, 1980.
- [3] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.
- [4] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936.
- [5] David Deutsch. Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London A*, 400:97–117, 1985.
- [6] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6/7):467–488, 1982.
- [7] Ian Glendinning and Bernhard Ömer. Parallelization of the general single qubit gate and CNOT for the QC-lib quantum computer simulator library. Technical Report, AURORA Technical Report TR2003-04, 2003.
- [8] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219. ACM, New York, 1996.
- [9] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.
- [10] Tyson Jones, Anna Brown, Ian Bush, and Simon C. Benjamin. QuEST and high performance simulation of quantum computers. *Scientific Reports*, 9:10736, 2019.

- [11] Alexei Y. Kitaev. Quantum measurements and the Abelian stabilizer problem. arXiv:quant-ph/9511026, 1995.
- [12] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.
- [13] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.
- [14] Qiskit contributors. Qiskit Aer: high-performance simulator for Qiskit. <https://qiskit.github.io/qiskit-aer/>, 2024. accessed May 2026.
- [15] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE Computer Society Press, 1994.
- [16] Mikhail Smelyanskiy, Nicolas P. D. Sawaya, and Alán Aspuru-Guzik. qHiPSTER: The quantum high performance software testing environment. arXiv:1601.07195, 2016.
- [17] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society, Series 2*, 42:230–265, 1936.
- [18] Lieven M. K. Vandersypen, Matthias Steffen, Gregory Breyta, Costantino S. Yannoni, Mark H. Sherwood, and Isaac L. Chuang. Experimental realization of Shor’s quantum factoring algorithm using nuclear magnetic resonance. *Nature*, 414:883–887, 2001.
- [19] William K. Wootters and Wojciech H. Zurek. A single quantum cannot be cloned. *Nature*, 299:802–803, 1982.